

تاکتوس

شماره نخست، تابستان ۱۴۰۱
انجمن علمی ACM دانشگاه اصفهان



वेवेवेल



هیئت تحریریه:

صاحب امتیاز:
انجمن علمی ACM دانشگاه اصفهان



مدیرمسئول:
محمدکاظم هرندی
دبیر سابق انجمن علمی ACM دانشگاه اصفهان



سرمدبیر:
زهرا معصومی
عضو سابق شورای مرکزی انجمن علمی ACM دانشگاه اصفهان



مشاور علمی این شماره:
دکتر آرشد شفیعی
عضو هیئت علمی دانشگاه اصفهان



واژه و طرح:

ویراستار علمی و ادبی:
پوریا طلائی



صفحه آرا و طراح اجرایی:
مهشاد خسروی



گرافیکست:
سارا رئوفی



«نشریه کاکتوس، گرافی برای اتصال ایده‌های شماست. هر نظر و پیشنهاد، یالی جدید در این ساختار است. برای همکاری و مشارکت در رشد این شبکه، از طریق بارکد با ما در ارتباط باشید.»

فهرست

۶

سرمقاله

۷

معرفی انجمن علمی ACM دانشگاه اصفهان
و نشریه کاکتوس

۱۰

ارائه‌ی راهکاری برای مصورسازی الگوریتم‌های
داده‌ساختار توسط زبان لاتک

۱۴

داده‌ساختارهای Retroactive

۲۰

تفکر ماشینی؛
از بازی تقلید تورینگ تا ناتمامیت گودل

۲۲

گراف‌ها و شبکه‌ها

۲۶

چیزهایی درمورد MEX

۲۹

کاکتوس پلاس پلاس



سرمقاله



دکتر بهروز شاهقلی قهفرخی (عضو هیئت علمی دانشکده مهندسی کامپیوتر دانشگاه اصفهان و استاد مشاور انجمن ACM)

انجمن علمی ACM دانشگاه اصفهان در سال ۱۳۸۸ با همت جمعی از دانشجویان کارشناسی و تحصیلات تکمیلی دانشکده‌ی مهندسی کامپیوتر به عنوان یک انجمن دانشجویی رسمی در دانشگاه ثبت گردید. لیکن فعالیت‌های دانشجویی در حوزه‌ی مسابقات ICPC که با محوریت انجمن جهانی ACM برگزار می‌شد، سابقه‌ی بیش‌تری دارد و به اواخر دهه‌ی هفتاد برمی‌گردد و مرحوم دکتر قوام‌نیا نقش موثری در ترویج آن و ترغیب دانشجویان به برگزاری مسابقات برنامه‌نویسی دانشگاهی و شرکت در مسابقات ICPC رسمی داشتند. از آن زمان تاکنون فعالیت‌های جنبی دانشجویان کارشناسی دانشکده‌ی مهندسی کامپیوتر دانشگاه اصفهان در حوزه‌ی الگوریتم و برنامه‌نویسی همواره شاهد رشد و پیشرفت بوده است و پس از شکل‌گیری انجمن علمی-دانشجویی ACM نیز، علاوه‌بر برگزاری و شرکت در مسابقات برنامه‌نویسی، فعالیت‌های گسترده‌تری برای ترویج مفاهیم و روش‌های حل مساله کارآمد، و شگردها و تکنیک‌های برنامه‌نویسی کارا برنامه‌ریزی و اجرا شده است. یکی از بسترهای ارزشمند برای ترویج این موارد و تقویت توانمندی‌های دانشجویان این رشته، مجلات علمی دانشجویی است. انتشار مقالات مختلف در راستای ترویج روش‌های نوین در برنامه‌نویسی و الگوریتم‌های جدید و کارآمد برای حل کامپیوتری مسائل پیچیده از اهداف مجله علمی-دانشجویی کاکتوس خواهد بود. انتشار اولین شماره از این مجله‌ی دانشجویی توسط مجموعه دست‌اندرکاران فعلی انجمن را به فال نیک گرفته و امیدوارم که شاهد تداوم انتشار و ارتقای محتوای آن باشیم.

معرضی انجمن علمی ACM دانشگاه اصفهان و نشریه کاکتوس



پوریا طلائی

دانشجوی مقطع کارشناسی مهندسی کامپیوتر دانشگاه اصفهان و
نائب‌دبیر سابق انجمن ACM



وبینارها و سمینارهای آموزشی متعدد از جمله گام‌های مهمی بوده است که در این دوره‌ی جدید برداشته شده‌اند. همچنین، مسابقاتی با هدف آشنا کردن ورودی‌های جدید با فضای مسابقات ICPC برگزار شده است تا آن‌ها نیز بتوانند هرچه زودتر با این فضا ارتباط برقرار کنند و مهارت‌های خود را در مسیر درستی پرورش دهند.

یکی از مهم‌ترین افتخاراتی که در این دوره نصیب انجمن علمی ACM دانشگاه اصفهان شده است، موفقیت چشمگیر تیم‌های برنامه‌نویسی اعزامی انجمن در مسابقات بین‌المللی ICPC است. در بیست‌وپنجمین دوره‌ی این مسابقات در منطقه‌ی غرب آسیا تیم‌های انجمن توانستند رتبه‌ی نهم تیمی و چهارم دانشگاهی را پس از دانشگاه‌های صنعتی شریف، تهران و صنعتی امیرکبیر کسب کرده و به مدال برنز دست پیدا کنند. این افتخار بزرگ حاصل تلاش، پشتکار و همدلی اعضای تیم‌های شرکت‌کننده است و نشان‌دهنده‌ی ظرفیت بالای دانشجویان دانشگاه اصفهان در رقابت‌های سطح بالا است.

انجمن ACM یا Association for Computing Machinery بزرگ‌ترین انجمن علمی و آموزشی در حوزه‌ی علوم کامپیوتر در جهان است که در سال ۱۹۴۷ تأسیس شد. این انجمن با هدف گسترش دانش و ارتقای مهارت‌های تخصصی در زمینه‌ی محاسبات و فناوری اطلاعات فعالیت می‌کند و با برگزاری کنفرانس‌ها، انتشار مقالات علمی و سازمان‌دهی مسابقات معتبر جهانی، نقشی کلیدی در پیشرفت علوم کامپیوتر ایفا کرده است. یکی از مهم‌ترین رویدادهای ACM، مسابقات بین‌المللی برنامه‌نویسی دانشجویی (ICPC) است که به‌عنوان یکی از معتبرترین مسابقات برنامه‌نویسی جهان شناخته می‌شود و دانشجویان مستعد را از سراسر دنیا گرد هم می‌آورد تا توانایی‌های خود را در حل مسائل الگوریتمی و بهینه‌سازی کد به چالش بکشند.

انجمن علمی ACM دانشکده‌ی مهندسی کامپیوتر دانشگاه اصفهان نیز یکی از فعال‌ترین انجمن‌های دانشگاهی در حوزه‌ی برنامه‌نویسی و علوم کامپیوتر است. پیشینه‌ی فعالیت‌های مرتبط با ACM در این دانشگاه به دهه‌ی ۷۰ بازمی‌گردد؛ زمانی که تیم‌های دانشجویی از طرف دانشگاه اصفهان در مسابقات برنامه‌نویسی منطقه‌ای شرکت می‌کردند. با افزایش این‌گونه فعالیت‌ها و همچنین تلاش‌های مسئولین مربوطه، در سال ۱۳۸۸ انجمن علمی ACM به طور رسمی تأسیس شد و از آن زمان تاکنون، این انجمن توانسته است نقش مهمی در توسعه‌ی مهارت‌های علمی و عملی دانشجویان ایفا کند.

در طول این سال‌ها، تیم‌های شرکت‌کننده از دانشگاه اصفهان موفق به کسب افتخارات متعددی شده‌اند که از جمله‌ی آن‌ها می‌توان به کسب رتبه‌ی سوم در مسابقات منطقه‌ای غرب آسیا و همچنین مقام سوم در بین تیم‌های تمام‌خانم در همین مسابقات اشاره کرد. این موفقیت‌ها، نتیجه‌ی سال‌ها تلاش و فعالیت دانشجویان علاقه‌مند و متعهدی است که همواره در راستای ارتقای سطح علمی خود و دانشگاهشان کوشیده‌اند.

فعالیت‌های انجمن ACM دانشگاه اصفهان تنها به شرکت در مسابقات محدود نمی‌شود. این انجمن همواره تلاش کرده است تا بستر مناسبی برای رشد و یادگیری دانشجویان ایجاد کند. در این راستا، چندین دوره‌ی مسابقات برنامه‌نویسی برای دانش‌آموزان برگزار شده است تا آن‌ها از سنین پایین با مفاهیم علوم کامپیوتر آشنا شوند. علاوه بر این، انجمن با برگزاری مسابقات تخصصی در حوزه‌ی هوش مصنوعی، زمینه‌ای برای یادگیری و رقابت در این حوزه‌ی پرکاربرد فراهم کرده است.

در سال‌های اخیر، فعالیت‌های انجمن علمی ACM دانشگاه اصفهان بیش از پیش گسترش یافته و تنوع بیشتری پیدا کرده است. برگزاری مسابقات برنامه‌نویسی در سطح کشوری مانند چارباگ و میزبانی



و اما در تازه‌ترین اقدام، این انجمن تصمیم گرفته است که نشریه‌ای علمی و آموزشی با نام «کاکتوس» منتشر کند.

چرا کاکتوس؟

نام «کاکتوس» برای نشریه‌ی انجمن علمی ACM دانشگاه اصفهان از یک ساختار خاص در علوم کامپیوتر الهام گرفته شده است. گراف کاکتوس (Cactus Graph) نوعی گراف است که در آن هر یال حداکثر در یک دور شرکت دارد. این ساختار در بسیاری از الگوریتم‌های بهینه‌سازی و مسائل مرتبط با شبکه‌های کامپیوتری کاربرد دارد.

نشریه‌ی کاکتوس با هدف پوشش مقالات علمی مرتبط با علوم کامپیوتر، الگوریتم و برنامه‌نویسی منتشر می‌شود. این نشریه بستری برای معرفی جدیدترین مفاهیم، تکنیک‌ها و چالش‌های حوزه‌ی علوم کامپیوتر فراهم می‌کند و به دانشجویان و علاقه‌مندان کمک می‌کند تا درک بهتری از روش‌های حل مسئله و طراحی الگوریتم‌ها پیدا کنند. از مقالات پژوهشی گرفته تا بررسی مسابقات برنامه‌نویسی و مصاحبه با چهره‌های شاخص این حوزه، کاکتوس تلاش دارد تا منبعی الهام‌بخش برای جامعه‌ی دانشگاهی باشد.

علاوه بر جنبه‌ی علمی، انتخاب نام «کاکتوس» یادآور ویژگی‌های منحصربه‌فرد این گیاه نیز هست. درست مانند یک کاکتوس که در سخت‌ترین شرایط رشد می‌کند، برنامه‌نویسان و دانشجویان علوم کامپیوتر نیز در مواجهه با چالش‌های پیچیده، راه‌حل‌هایی خلاقانه می‌یابند. این نشریه می‌خواهد همراهی برای دانشجویان در این مسیر باشد و به آن‌ها در مسیر یادگیری و پیشرفت کمک کند.





ارائه‌ی راهکاری برای مصورسازی الگوریتم‌های داده‌سافتار توسط زبان لاتک



یاسمین اکبری

دانشجوی مقطع کارشناسی مهندسی کامپیوتر دانشگاه اصفهان

همان‌طور که در تصویر صفحه بعد مشاهده می‌کنید، نمای کلی پروژه به این صورت است که ما کتابخانه‌ای در زبان پایتون ایجاد کردیم که درواقع شامل مجموعه‌ای از کدهای پایتونی برای مصورسازی الگوریتم‌های مختلف می‌باشد. عملکرد کتابخانه به این صورت است که پس از فراخوانی کتابخانه و الگوریتم مورد نظر از آن، باید مواردی را به عنوان ورودی برنامه، مشخص کنیم. ورودی‌های مورد نظر شامل:

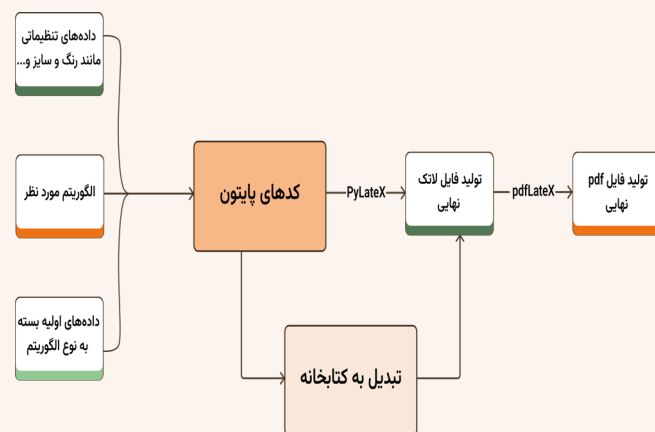
(۱) مشخص کردن داده‌های اولیه بسته به نوع الگوریتم فراخوانی شده (مثلاً برای یک الگوریتم مرتب‌سازی باید داده‌های مربوط به آرایه مشخص شوند).

(۲) مشخص کردن سایز، ابعاد و رنگ‌هایی که کاربر از تصاویر خروجی انتظار دارد.

در ادامه پس از تعیین ورودی‌های لازم، در خروجی فایلی به زبان لاتک تولید خواهد شد که با اجرای آن تصاویر مرحله به مرحله اجرای الگوریتم را به صورت گام به گام خواهیم داشت.

در دنیای امروز، الگوریتم‌ها به‌عنوان قلب تپنده‌ی محاسبات و پردازش‌های کامپیوتری شناخته می‌شوند. از مرتب‌سازی داده‌ها و جست‌وجوی اطلاعات تا مدیریت منابع در سیستم‌های چندکاربره، الگوریتم‌ها نقشی کلیدی در بهینه‌سازی و اجرای وظایف مختلف دارند. به دلیل اهمیت بالای الگوریتم‌ها، درک عمیق و دقیق عملکرد آن‌ها برای دانشجویان، پژوهشگران و برنامه‌نویسان امری ضروری است. همچنین یکی از چالش‌های اصلی در آموزش و یادگیری الگوریتم‌ها، فهم دقیق نحوه‌ی عملکرد هر الگوریتم در مراحل مختلف است. این پروژه با ارائه‌ی تصاویر و نمودارهای گام‌به‌گام از فرایند اجرا و عملکرد الگوریتم‌ها، به مخاطبان کمک می‌کند تا به‌صورت تعاملی و بصری، نحوه عملکرد هر الگوریتم را مشاهده و درک کنند.

پژوهش حاضر، با هدف پیاده‌سازی و مصورسازی طیف گسترده‌ای از الگوریتم‌های داده‌ساختار در زبان برنامه‌نویسی پایتون توسعه یافته است. این پژوهش نه تنها شامل پیاده‌سازی الگوریتم‌های مختلفی مانند الگوریتم‌های جست‌وجو، مرتب‌سازی، مدیریت لیست‌های پیوندی، انواع درخت، استک و صف است، بلکه با تولید خودکار فایل‌های لاتک، فرایند اجرای هر الگوریتم را به‌صورت بصری و گرافیکی نمایش می‌دهد.



افزودن الگوریتم‌های مربوط به گراف‌ها:

با اضافه کردن الگوریتم‌های مربوط به گراف‌ها به پروژه، می‌توان دامنه‌ی کاربردهای آن را به طور قابل توجهی گسترش داد. الگوریتم‌های کوتاه‌ترین مسیر مانند دایکسترا و الگوریتم‌های مربوط به درخت‌های پوشا مانند الگوریتم کراسکال و پرایم، می‌توانند به کتابخانه اضافه شوند. این افزودنی‌ها به کاربران این امکان را می‌دهند که تحلیل‌های گسترده‌تری از ساختار گراف‌ها انجام دهند و مسائل پیچیده‌تر را مدل‌سازی کنند.

امکان دریافت ورودی به صورت فایل:

افزودن قابلیت دریافت ورودی به صورت فایل می‌تواند فرآیند استفاده از پروژه را تسهیل کند. با این ویژگی، کاربران قادر خواهند بود تا داده‌ها و پارامترهای موردنظر خود را از فایل‌های متنی بارگذاری کنند و نتایج تحلیل‌های الگوریتمی را به راحتی مشاهده کنند. این قابلیت می‌تواند به ویژه برای کاربرانی که با داده‌های بزرگ و پیچیده سر و کار دارند، بسیار مفید باشد.

تولید خروجی تصاویر متحرک:

علاوه بر تولید خروجی لاتک، امکان تولید تصاویر متحرک از مراحل مختلف الگوریتم‌ها می‌تواند به درک بهتر و جذاب‌تر نتایج کمک کند. تصاویر متحرک می‌توانند فرایندهای الگوریتمی را به صورت متوالی و در حال حرکت نمایش دهند که به ویژه برای آموزش و ارائه‌های بصری بسیار مفید است.

ایجاد امکان پیاده‌سازی مشترک الگوریتم‌ها:

به منظور افزایش انعطاف‌پذیری و کارایی، می‌توان امکان فراخوانی تعدادی از الگوریتم‌ها به صورت مشترک از طریق تعریف یک تابع جدید را فراهم کرد. این ویژگی می‌تواند به کاربران این امکان را بدهد که چندین الگوریتم را به طور هم‌زمان پیاده‌سازی کنند و نتایج را به صورت یکپارچه مشاهده کنند. برای مثال، ترکیب الگوریتم‌های مرتب‌سازی با الگوریتم‌های جست‌وجو یا ترکیب الگوریتم‌های گرافی با الگوریتم‌های پردازش داده، می‌تواند کاربردهای جدیدی را ایجاد کند.

امکان هماهنگی با انواع نوع و سایز صفحات:

برای افزایش رضایت کاربران افزودن امکان هماهنگ‌سازی تصاویر خروجی با صفحات A5 و Beamer نیز می‌تواند مفید باشد.

با پیاده‌سازی این پیشنهادها، پروژه می‌تواند به ابزاری جامع‌تر و قدرتمندتر تبدیل شود که به نیازهای متنوع‌تری از کاربران پاسخ دهد و قابلیت‌های تحلیل و مصورسازی الگوریتمی را به سطح بالاتری ارتقا دهد.

یکی از ویژگی‌های کلیدی این پروژه، قابلیت دریافت داده‌ها و تنظیمات مختلف بر اساس نیاز کاربران است. کاربران می‌توانند داده‌ها، ابعاد، رنگ‌ها و ویژگی‌های بصری را به دلخواه خود تنظیم کنند. این ویژگی به آن‌ها این امکان را می‌دهد که نتایج را به صورت متناسب با نیازهای خاص خود مشاهده کرده و فرایندهای الگوریتمی را به شکل قابل فهم و شخصی‌سازی شده دریافت نمایند.

در واقع همان‌طور که پیش‌تر اشاره شد این پروژه با هدف پیاده‌سازی و مصورسازی الگوریتم‌های مختلف، توانسته است ابزار مفیدی برای تحلیل و بررسی الگوریتم‌ها ارائه دهد. با استفاده از ترکیب زبان برنامه‌نویسی پایتون و لاتک، پروژه موفق شده است تصاویری شفاف و قابل درک از فرایندهای الگوریتمی ارائه دهد که می‌تواند به طور مؤثری در زمینه‌های مختلف آموزشی و تحقیقاتی مورد استفاده قرار گیرد. در نهایت، می‌توان گفت این پروژه با تمرکز بر تولید مستندات بصری و تعاملی با امکان شخصی‌سازی تصاویر مطابق با نیاز و سلیقه کاربر و استفاده از ابزارهای پیشرفته برای مصورسازی الگوریتم‌ها، به طور منحصر به فردی از سایر پروژه‌های مشابه، متمایز می‌شود.

پیشنهادهایی برای ادامه پژوهش

با توجه به ظرفیت‌های گسترده، جذابیت و کارایی این پروژه، امکانات وسیعی برای توسعه و گسترش آن وجود دارد. این پروژه با ارائه راهکاری قدرتمند برای پیاده‌سازی و مصورسازی الگوریتم‌ها، نقطه‌ی شروعی برای ارائه قابلیت‌های جدید و بهبودهای بیشتر است. به منظور ارتقای این پروژه و بهره‌برداری بهتر از توانمندی‌های آن، در ادامه پیشنهادهایی ارائه خواهد شد که می‌تواند به گسترش دامنه‌ی کاربرد، افزایش کارایی و بهبود تجربه‌ی کاربری پروژه کمک کنند.

راههای دسترسی به کتابخانه

۱. به صورت محلی و با دسترسی مستقیم به منبع کتابخانه:

github.com/ui-ce/algorithm-visualizer

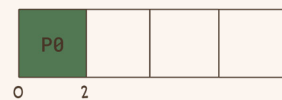
۲. به صورت نصب پکیج از اینترنت:

test.pypi.org/simple/algorithm-visualization-library

نمونه‌هایی از تصاویر تولید شده با استفاده از این کتابخانه:

الگوریتم زمان‌بندی کوتاه‌ترین کار:

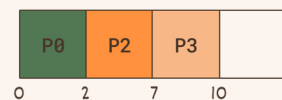
I.1 Time 0: Process P0



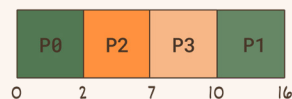
I.2 Time 2: Process P2



I.3 Time 7: Process P3



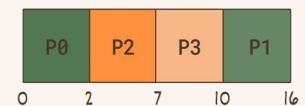
I.4 Time 10: Process P1



I.5 Final Process Table

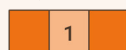
Process	Arrival Time	Burst Time	Service Time
P0	0	2	0
P1	1	6	10
P2	2	5	2
P3	3	3	7

I.6 Execution Order



الگوریتم ایجاد لیست پیوندی دوطرفه:

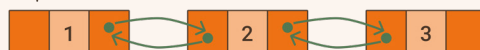
Step 1



Step 2



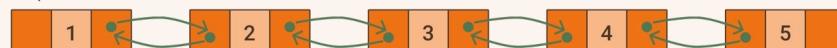
Step 3



Step 4



Step 5



الگوریتم پیمایش سطحی در درخت جستجوی دودویی:

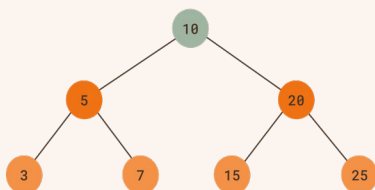


Figure 1: Step 1

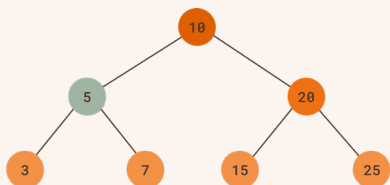


Figure 2: Step 2

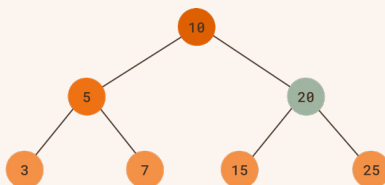


Figure 3: Step 3

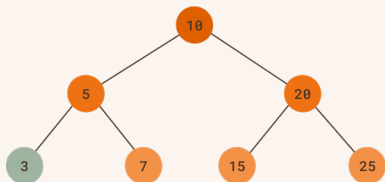


Figure 4: Step 4

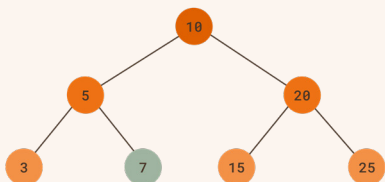


Figure 5: Step 5

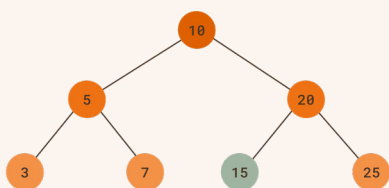


Figure 6: Step 6

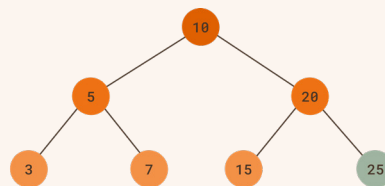
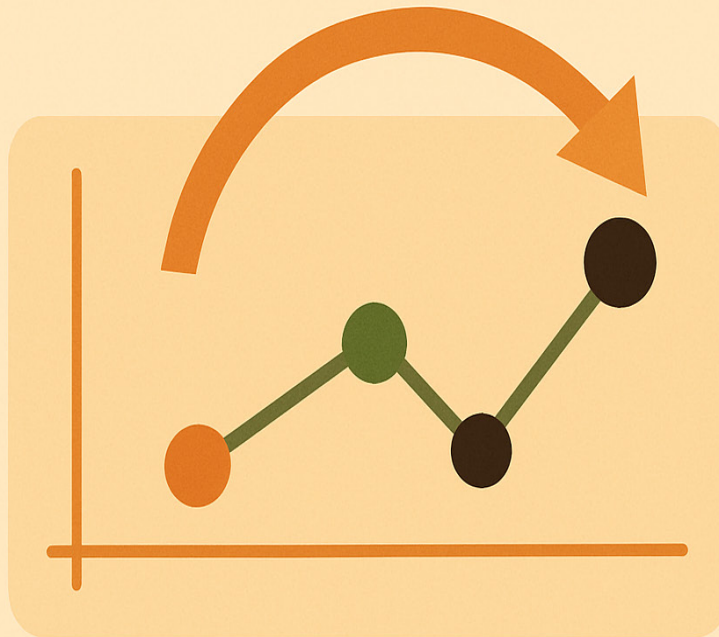


Figure 7: Step 7



داده‌ساختارهای Retroactive



مهدی حق‌وردی

دانشجوی مقطع کارشناسی مهندسی کامپیوتر دانشگاه اصفهان

عوض کردن گذشته

امروز قراره که گذشته رو عوض کنیم!
درسته!

در حالت کلی ما نمی‌تونیم گذشته رو عوض کنیم. مثلاً شما ممکنه برید به گذشته و پدربزرگتون رو بکشید، اما وقتی که اون رو کشتید، تاثیری که این اتفاق روی آینده می‌گذاره چیه و چقدره؟ آیا وقتی اون رو کشتید باعث نمی‌شه که دیگه پدرتون به دنیا نیاد و ازدواج نکنه و شمایی هم دیگه وجود نداشته باشید که به گذشته برید و پدربزرگ خودتون رو بکشید؟ این اتفاق اسمش پارادوکس پدربزرگه.

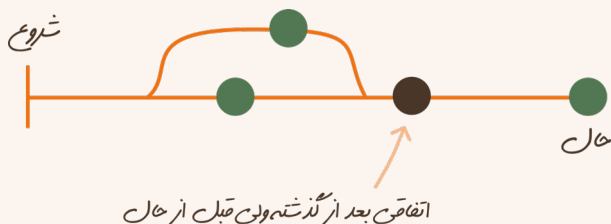
اما ما می‌تونیم گذشته رو توی دنیای کامپیوتر و در داده‌ساختارها عوض کنیم و با رعایت یکپارچگی داده (که جلوتر توضیح می‌دم چیه) بتونیم در گذشته تغییراتی ایجاد کنیم و نتیجه‌ش رو توی زمان حال ببینیم و یا حتی یکمی جلوتر از زمانی که عوض کردیم و یکمی عقب‌تر از زمان حال رو query کنیم و نتیجه‌ی تغییرات گذشته بر باقی داده‌ساختار رو ببینیم.

تفاوت بین داده‌ساختارهای persistent و retroactive

داده‌ساختارهای persistent داده‌ساختارهای معمولی ما هستن که هر روزه از اون‌ها استفاده می‌کنیم. این داده‌ساختارها چیزی رو راجع به زمان operation‌های مختلف ذخیره نمی‌کنند و اگه بخوایم در گذشته‌شون تغییر ایجاد کنیم، باید یک شاخه جدید ازشون بسازیم و به موقعیت حال دومی برسیم. نکته‌ی دیگری هم که این داده‌ساختارها دارن اینه که وقتی گذشته رو عوض می‌کنیم باید از همون گذشته دوباره کل زمان رسیدن به حال رو طی کنیم تا ببینیم چه اتفاقی می‌افته.

فرق بین داده‌ساختارهای partially retroactive و fully retroactive

فرق بین این دو خیلی ساده‌ست، توی مثال‌های بالا اگه فقط اینکه چه اتفاقی برای «الان» می‌افته برای ما مهم باشه و نخواهیم گذشته‌ی نزدیک‌تری رو ببینیم، ما یه داده‌ساختار نیمه‌رترواکتیو می‌خواهیم. اما اگه برای ما مهم باشه که نه من می‌خواهم گذشته‌ی نزدیک به زمان حال رو هم query کنم باید داده‌ساختارمون کاملاً رترواکتیو باشه.

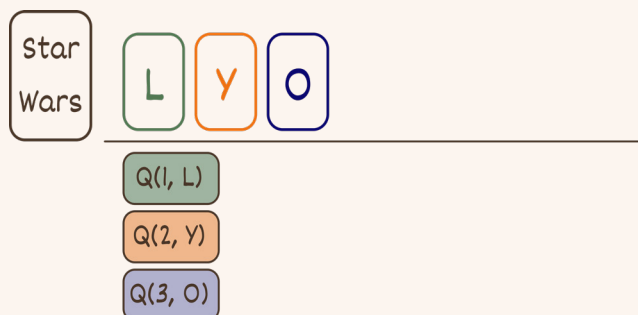


داده‌ساختارهای رترواکتیو کدوما هستن؟

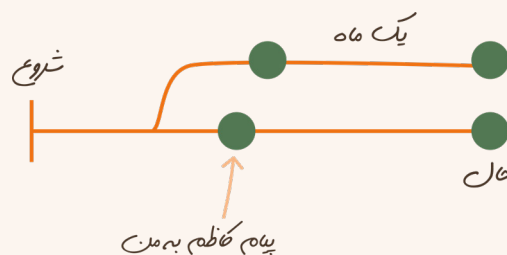
بحثی که اینجا هست اینه که ما عبارت «داده‌ساختار رترواکتیو» رو به اون صورت نداریم، بلکه ما می‌تونیم چیزایی به داده‌ساختارهای کنونی‌مون اضافه کنیم و اون‌ها رو رترواکتیو کنیم. برای مثال بیایم به یک queue یه سری موارد اضافه کنیم و اون رو رترواکتیو کنیم و ازش استفاده کنیم.

صف رترواکتیو

بیاید یک صفی رو پشت گیشه‌ی سینمایی تصور کنیم که افراد مختلفی می‌خوان قسمت سوم از Prequel Trilogy عه استاروارز به اسم Revenge Of the Sith رو ببینن. اضافه شدن به صف رو با $Q(\text{time}, \text{who})$ و کم شدن از صف رو با $D(\text{time})$ نشون می‌دیم.

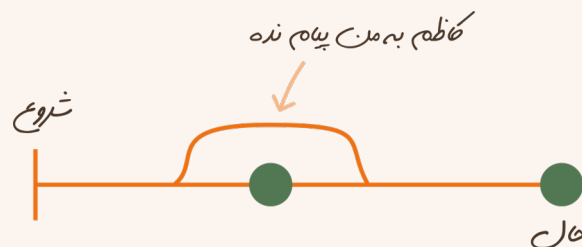


در این شکل سه نفر در زمان‌های ۱، ۲ و ۳ وارد صف سینما شدن. بیاید یک بار پر و خالی شدن صف رو پشت سر هم ببینیم.

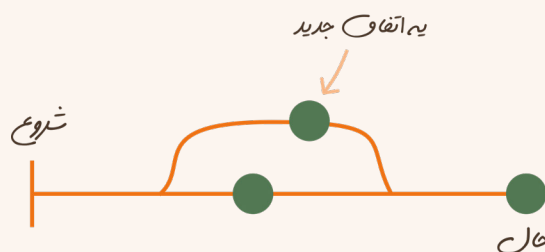


همین‌طور که توی شکل می‌بینیم کاظم به من در گذشته پیام داده و راجع به نشریه با من حرف زده و من قبول کردم که «همین مقاله» رو بنویسم. حالا بیاید فکر کنیم اگه کاظم من رو یک روز دیگه توی فنی می‌دید و باهام حرف می‌زد یا مثلاً اگه من قبول نمی‌کردم، چه اتفاقی می‌افتاد؟ البته این واضحه که این مقاله وجود نداشت ولی من درست در همین ساعتی که دارم مقاله رو می‌نویسم داشتم چه کاری می‌کردم؟ اگه این رو با یه داده‌ساختار persistent می‌خواستیم مدل کنیم باید برمی‌گشتیم قبل عید و مثلاً من قبول نمی‌کردم و می‌اومدیم جلو ببینیم چی میشه. اما این قضیه با داده‌ساختارهای retroactive فرق می‌کنه.

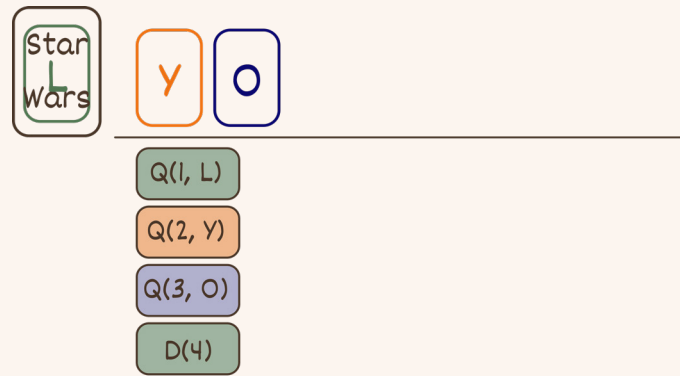
این داده‌ساختارها مثل این شکل به ما اجازه می‌دن که یه کاری رو توی گذشته انجام ندیم و ببینیم زمان حال چه اتفاقی براش می‌افته ولی لازم نباشه یه دنیای دیگه خلق کنیم.



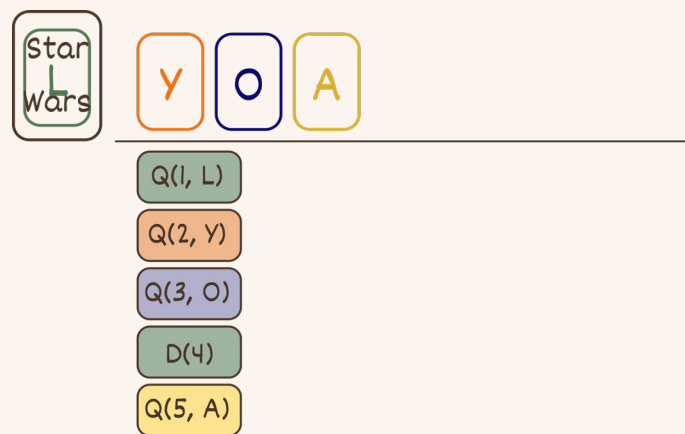
یا مثلاً یه کاری به گذشته اضافه کنیم و ببینیم چه اتفاقی برای زمان حال می‌افته.



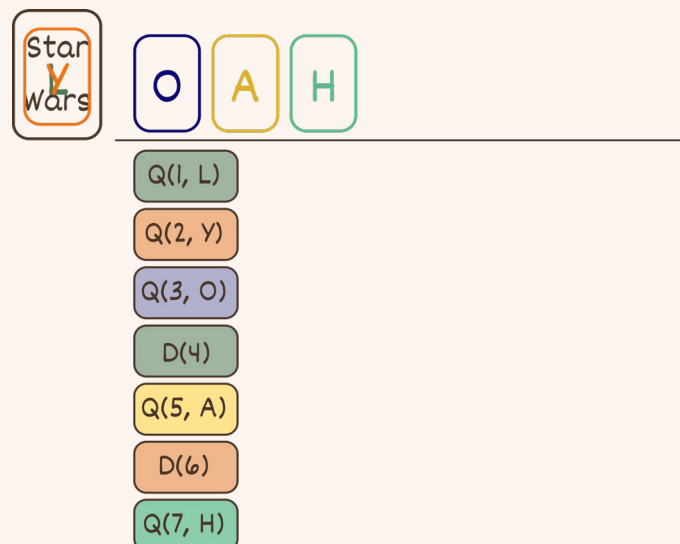
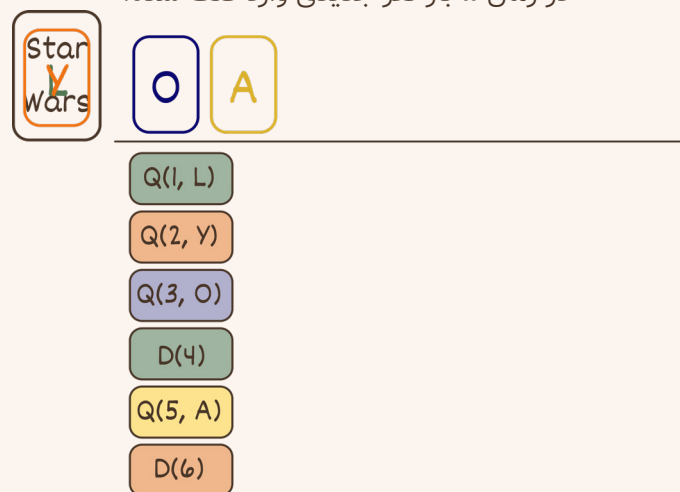
مثلاً وقتی که کاظم به من پیام داد من پیامش رو سبن نمی‌زدم یا اگه می‌زدم جوابی بهش نمی‌دادم چه اتفاقی می‌افتاد؟ آیا اون دیگه بهم چیزی نمی‌گفت؟ آیا دوباره بهم پیام می‌داد؟



در زمان ۴، یک D انجام شده و نفر اول صف وارد سینما شد.



در زمان ۵ باز نفر جدیدی وارد صف شده.





A H

Q(1, L)
Q(2, Y)
Q(3, O)
D(4)
Q(5, A)
D(6)
Q(7, H)
D(8)



H

Q(1, L) D(9)
Q(2, Y)
Q(3, O)
D(4)
Q(5, A)
D(6)
Q(7, H)
D(8)



H C T G

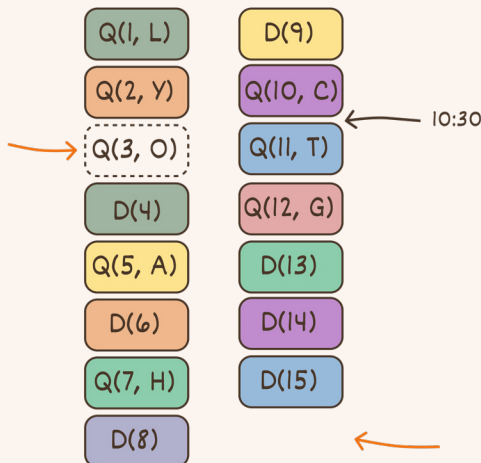
Q(1, L) D(9)
Q(2, Y) Q(10, C)
Q(3, O) Q(11, T)
D(4) Q(12, G)
Q(5, A)
D(6)
Q(7, H)
D(8)



Q(1, L)	D(9)
Q(2, Y)	Q(10, C)
Q(3, O)	Q(11, T)
D(4)	Q(12, G)
Q(5, A)	D(13)
D(6)	D(14)
Q(7, H)	D(15)
D(8)	D(16)

و حالا هم صف ما خالی شده.

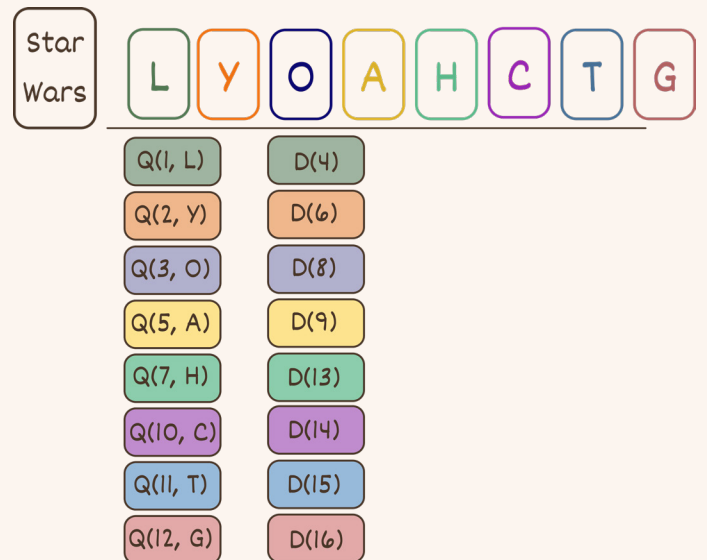
Star Wars



حالا بیاید در گذشته تغییری ایجاد کنیم. بیاید فرض کنیم که وضعیت صف ما اینه:

فرض کنیم که نفر سوم در زمان ۳ داخل صف وارد نشد (اگه ما یکی از Qها کم کنیم منطقا کسی رو نداریم که D کنیم پس یکی از Dها رو هم از آخر صف باید برداریم تا یکپارچگی داده حفظ بشه)، آیا میتونید خیلی سریع وضعیت صف رو در زمان ۱۰:۳۰ به من بگید؟ در ضمن برای جواب دادن به این سوال هم فقط از داده‌هایی که الان داریم باید استفاده کنیم ولی باید بتونیم سریع محاسبه انجام بدیم.

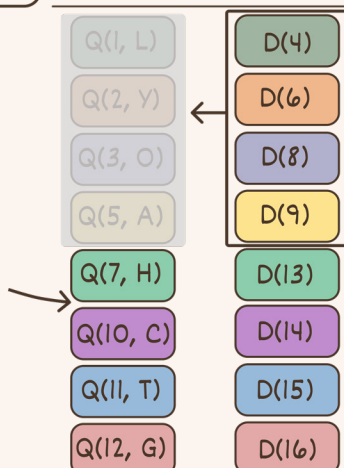
بیاید به این داده‌ها مون یک جور دیگه نگاه کنیم.



تا وقتی که یکپارچگی داده‌ی ما حفظ بشه (یعنی اگه یه D بهشون اضافه کردیم که فردی رو از سر صف برمی‌داره، حتما یک Q هم اضافه کنیم که واقعا فردی توی صف باشه که برش داریم. یا به صورت دقیق‌تر تعداد Dهای ما باید حتما کوچک‌تر مساوی تعداد Qهای ما باشه چون اگه برعکس باشه ما کسی رو از بالای صف برمی‌داریم که نیست و دلیل اینکه ما توی دنیای واقعی نمی‌تونیم رفتار رترواکتیو داشته باشیم همینه که همه‌چیز یکپارچه نخواهد بود) ما می‌تونیم به این اتفاقات به صورت کاملا مستقل از هم نگاه کنیم.

اگه توی زمان جلو بیایم، در زمان ۳، سه نفر در صف هستن و در زمان ۴ یک نفر از سر صف وارد سینما می‌شه که اولین ورودی لیست Qهای ماست. در زمان ۵ یک نفر وارد صف می‌شه، و در زمان ۶ یک نفر خارج می‌شه که همون نفر دوم صف ماست و...

Star Wars



حالا بیاید به صورت retroactive به سوالمون جواب بدیم. سوال این بود که اگه ما سومین Q رو از گذشته حذف کنیم، وضعیت صف در ساعت ۱۰ و نیم چه شکلیه؟

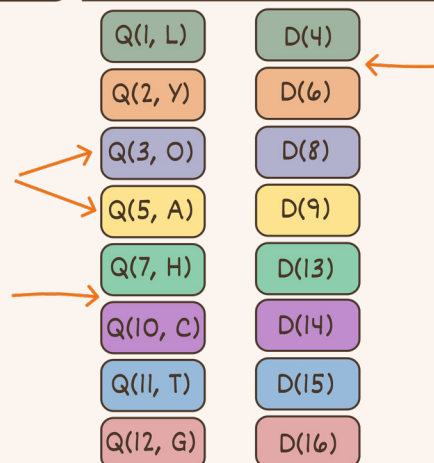
اگه به شکل دقت کنیم، ما در ساعت ۱۰ و نیم به لیست Dها مون نگاه می‌کنیم و می‌بینیم چند تا D انجام شده، و به همون تعداد از لیست Qها مون خط می‌زنیم و به وضوح و خیلی سریع می‌بینیم که در ساعت ۱۰ و نیم فقط C توی صف ایستاده بوده.

دیدید چه سریع بود؟ بیاید به مثال دیگه هم بزنیم.

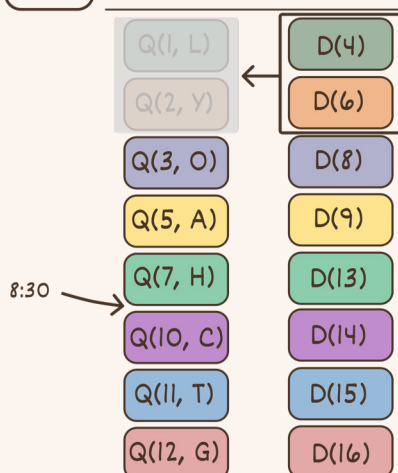
Star Wars

من ترتیب ورود نفرهای A و O رو عوض کردم، ورود نفر C رو هم حذف کردم و عملیات D در زمان ۶ رو هم پاک کردم. حالا سوال من اینه، در ساعت ۸:۳۰ نفر دوم صف کیه؟

بیاید تا با این دو لیست مون دقیقاً عین راه حل قبلی بهش جواب بدیم.



Star Wars



ما بررسی می‌کنیم که تا ساعت ۸:۳۰ چند بار عملیات D داشتیم و می‌بینیم که دو بار این اتفاق افتاده؛ پس دو نفر از لیست Q رو خط می‌زنیم و می‌بینیم که آقای O که در زمان ۵ اومده توی صف و نفر دومه و داره به ما میگه Hello There (اونایی که میدونن!) و آقای O دو جواب ماست.

برای نگه داشتن و دسترسی سریع به این عملیات‌ها ما باید از یک درخت جستجوی باینری خاص به اسم Order Statistics Tree استفاده کنیم. فرق این درخت با درخت جستجوی باینری در اینه که

BBST:

Search in $O(\log n)$
Insert in $O(\log n)$
Delete in $O(\log n)$

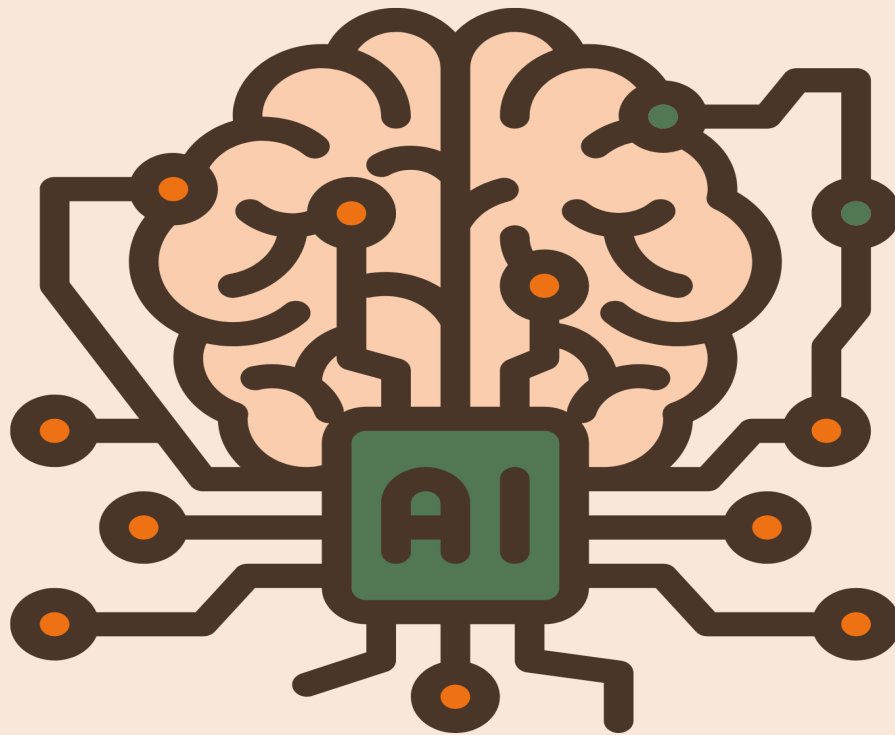
Order Statistics Tree:

Search in $O(\log n)$
Insert in $O(\log n)$
Delete in $O(\log n)$
Order-of-Key $O(\log n)$
Find-by-Order in $O(\log n)$

در واقع درخت ترتیب آماری (چه معادل جالبی!) دو عملیات بیشتر از BBST داره که اون‌ها می‌تونن ترتیب اون برگ رو در درخت در زمان $O(\log n)$ پیدا کنن و می‌تونیم بگیم سومین برگ رو بهم بده که این در زمان $O(\log n)$ انجام می‌شه. اون دو لیستی که بالاتر دیدیم در واقع هر کدوم یک درخت جدا هستن و هر عملیات Q یا D یک برگ از این درخت محسوب می‌شن و جستجو، اضافه کردن و حذف کردن مثل یک درخت باینری انجام می‌شه ولی بیاید کاربرد دو عملیات اضافه‌شون رو بهتون با جواب‌هایی که نوشتیم، نشون بدم.

اگه از پاسخ قبلی یادتون باشه، ما می‌خواستیم ببینیم تا ساعت ۸ چند بار عملیات D انجام شده، خب ما از این درخت می‌خوایم که index اولین عملیات بعد از ساعت هشت رو به ما بده. که می‌شه D(۹) و ایندکسش هست ۲؛ این یعنی قبل از من دو عملیات دیگه هستن. پس ما باید از درخت Q‌ها دو برگ اول رو برداریم و به برگ سوم برسیم و متوجه می‌شیم که نفر اول صف ما چه کسی هست.^۱

۱. اونایی که میدونن!



تفکر ماشینی؛ از بازی تقلید تورینگ تا ناتمامیت گودل



فاطمه رضائی

دانشجوی مقطع کارشناسی علوم کامپیوتر دانشگاه اصفهان

و تعریف شده است. آنها دقیقاً طبق برنامه‌ای که برایشان نوشته شده عمل می‌کنند، هرچند ممکن است در این برنامه‌ریزی عناصر تصادفی هم گنجانده شده باشد.

واژه تفکر را از دیدگاه دو دانشمند بزرگ حوزه علوم کامپیوتر و ریاضی، آلن تورینگ و گودل بررسی می‌کنیم. آلن تورینگ به مفاهیم انتزاعی تفکر مانند آگاهی یا درک اشاره نمی‌کند بلکه معتقد است اگر یک ماشین بتواند در تعاملات به گونه‌ای رفتار کند که از انسان غیرقابل تشخیص باشد، در واقع می‌توان گفت که ماشین توانایی تفکر دارد. بنابراین از دیدگاه او، فکر کردن همان توانایی انجام محاسبات منطقی و نمایش رفتار هوشمندانه است. تورینگ برای بررسی مفهوم «تفکر»، بازی‌ای به نام «بازی تقلید» را مطرح می‌کند. در این بازی، یک مرد، یک زن، و یک پرسش‌گر که ممکن است زن یا مرد باشد، حضور دارند. این افراد در دو اتاق جداگانه هستند و هدف پرسش‌گر این است که از طریق پرسیدن سوالات، تشخیص دهد کدام یک مرد و کدام زن است. هدف مرد این است

هر یک از ما در سال‌های اخیر بیش از پیش شاهد تغییرات ناشی از هوش مصنوعی و ماشین‌ها بوده‌ایم و این پیشرفت‌ها از دیرباز همواره سوالاتی را در ذهن انسان به وجود آورده‌اند. شاید یکی از مهم‌ترین این پرسش‌ها این باشد که آیا ماشین می‌تواند فکر کند؟ برای یافتن پاسخ ابتدا باید یک قدم عقب‌تر برویم، در واقع نخست باید بدانیم که منظور ما از ماشین و فکر کردن آن چیست.

در گام اول به تعریف ماشین می‌پردازیم. ماشین وسیله‌ای است که توسط انسان طراحی و ساخته می‌شود و بر اساس قوانین از قبل تعیین شده عمل می‌کند بنابراین ما انسان‌ها ماشین نیستیم، چرا که توسط انسانی دیگر ساخته نشده‌ایم. از دیدگاه تورینگ ماشین‌ها از چهار جزء اصلی تشکیل شده‌اند: حافظه برای ذخیره‌ی اطلاعات، واحد محاسباتی برای انجام عملیات پایه، واحد کنترل که قوانین را اجرا می‌کند، و مکانیزم‌های ورودی و خروجی. نکته‌ی مهم این است که این تعریف مستقل از فناوری ساخت است و تفاوتی ندارد ماشین شیمیایی، الکتریکی یا مکانیکی باشد چرا که همگی در چارچوب این تعریف جای می‌گیرند. ماشین‌های واقعی حافظه‌ی محدودی دارند، اما مدل نظری تورینگ دارای حافظه نامحدود است. ویژگی کلیدی ماشین‌ها رفتار مبتنی بر قوانین مشخص

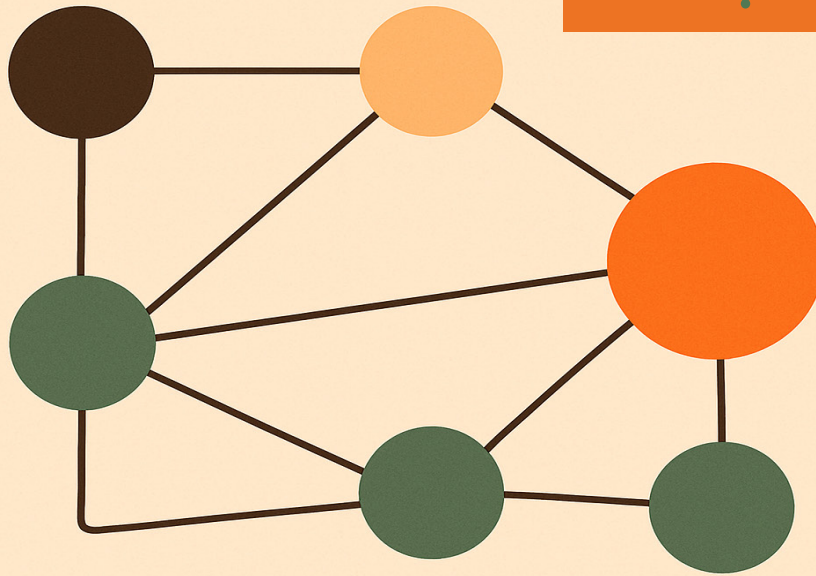
که سعی کند پاسخ‌هایی بدهد که پرسش‌گر را گمراه کند و او نتواند تشخیص دهد که کدام مرد است. بر اساس عقیده‌ی تورینگ ممکن است پرسش‌گر نتواند به راحتی تشخیص دهد که فرد در اتاق دیگر انسان است یا ماشین، چرا که ماشین می‌تواند همانند انسان پاسخ‌هایی بر اساس دستورات معینی بدهد.

در مقابل، کورت گودل با اتکا به قضیه‌ی ناتمامیت خود، محدودیت‌های ذاتی این سیستم‌ها را نشان می‌دهد. این قضیه اثبات می‌کند که در هر سیستم منطقی سازگار و به اندازه کافی قوی، همواره گزاره‌های درستی وجود دارند که درون آن سیستم غیرقابل اثبات هستند. گودل با تحلیل این نتیجه، به تفاوت بنیادین و اساسی بین ذهن انسان و ماشین‌های محاسباتی اشاره می‌کند. از نظر او، در حالی که ماشین‌ها محدود به قوانین و تعاریفات هستند، ذهن انسان توانایی درک مستقیم و شهودی حقایق را دارد.

گودل در سال ۱۹۳۱ با استفاده از نظریه‌ی اعداد نشان داد اگر فرض کنیم سیستم T ، مجموعه‌ای از اصول موضوعه و قواعد منطقی باشد که قادر به بیان محاسبات پایه‌ی ریاضی است، با روش کدگذاری گودلی هر گزاره، اثبات و فرایند استنتاج را به عددی طبیعی نسبت دهد؛ سپس، یک گزاره مانند ST را می‌سازد که به‌طور ضمنی می‌گوید: «این گزاره در سیستم T اثبات‌پذیر نیست.» اگر ST قابل اثبات باشد، سیستم T ناسازگار می‌شود چون گزاره‌ای اثبات شده که می‌گوید قابل اثبات نیست. اما اگر ST قابل اثبات نباشد، ولی درستی‌اش پذیرفته شود، آنگاه T ناقص است، چون یک گزاره درست را نمی‌تواند اثبات کند. طبق این قضیه، همیشه گزاره‌هایی وجود دارند که درست هستند ولی ماشین نمی‌تواند آن‌ها را اثبات یا حتی تشخیص دهد. مثلاً ماشین ممکن است نتواند درستی گزاره‌ای را اثبات کند، اما انسان با شهود و درک خود قادر به فهم درستی آن و در عین حال غیرقابل اثبات بودنش است.

بنابراین همه‌چیز به تعریف ما از تفکر برمی‌گردد. ماشین‌ها فقط در محدوده‌ی «محاسبه‌پذیر» باقی می‌مانند. در حالی که تفکر انسان شامل جنبه‌های غیرمحاسبه‌پذیر نیز هست. در واقع، ماشین‌ها قادر به انجام عملیات پیچیده هستند، ولی نمی‌توانند همچون انسان‌ها به درک و شهود دست یابند که فراتر از منطق و قواعد هستند. این قابلیت ویژه‌ی انسان است که می‌تواند اصول و قوانین جدیدی کشف کند که ماشین‌ها قادر به انجام آن نیستند. بنابراین، پاسخ به سوال «آیا ماشین‌ها می‌توانند فکر کنند؟» حداقل طبق تعریف گودل این است که خیر، ماشین‌ها نمی‌توانند به همان معنای انسان‌ها تفکر کنند.

گراف‌ها و شبکه‌ها



در دنیای پیچیده‌ی امروز، مفاهیم انتزاعی ریاضی می‌توانند کلید حل بسیاری از مسائل واقعی باشند. یکی از این مفاهیم قدرتمند، گراف‌ها هستند. شاید در نگاه اول، گراف‌ها صرفاً مجموعه‌ای از نقاط و خطوط به نظر برسند، اما در واقعیت، آن‌ها ابزاری فوق‌العاده برای مدل‌سازی و تحلیل ارتباطات و روابط بین اشیاء هستند. از شبکه‌های اجتماعی که روابط دوستانه را نشان می‌دهند تا شبکه‌های حمل‌ونقل که مسیرهای جاده‌ای را نمایش می‌دهند، گراف‌ها در همه جا حضور دارند.

گراف چیست؟

به زبان ساده، یک گراف از دو بخش اصلی تشکیل شده است: راس‌ها (nodes) و یال‌ها (edges). راس‌ها نشان‌دهنده‌ی اشیاء یا موجودیت‌ها هستند، در حالی که یال‌ها بیانگر روابط یا اتصالات بین این اشیاء می‌باشند. **راس‌ها:** راس‌ها را می‌توان به صورت نقاطی در نظر گرفت که موجودیت‌های مورد نظر ما را نشان می‌دهند. به عنوان مثال، در یک شبکه‌ی اجتماعی، هر شخص می‌تواند یک راس باشد.

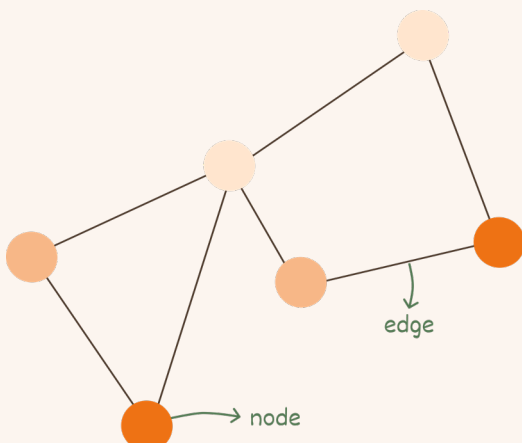
یال‌ها: یال‌ها خطوطی هستند که دو راس را به هم متصل می‌کنند و نشان‌دهنده‌ی رابطه‌ی بین آن‌ها هستند. در مثال شبکه‌ی اجتماعی، یک یال می‌تواند نشان‌دهنده‌ی دوستی بین دو شخص باشد. گراف‌ها می‌توانند جهت‌دار یا بی‌جهت باشند. در گراف‌های بی‌جهت، یال‌ها رابطه‌ی دوطرفه را نشان



محمد مزروعی

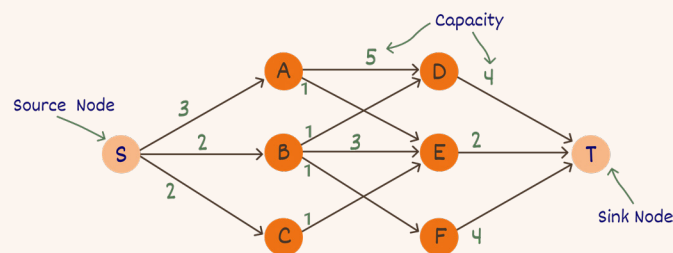
دانشجوی مقطع کارشناسی مهندسی کامپیوتر دانشگاه اصفهان
بازبینی علمی و محتوایی: متین اعظمی

می‌دهند (مانند دوستی در شبکه‌ی اجتماعی). اما در گراف‌های جهت‌دار، یال‌ها رابطه‌ی یک‌طرفه را مشخص می‌کنند (مثلاً در یک شبکه‌ی جاده‌ای، خیابان‌های یک‌طرفه). همچنین، یال‌ها می‌توانند وزن‌دار باشند، به این معنی که به هر یال یک مقدار (وزن) نسبت داده می‌شود که می‌تواند نشان‌دهنده‌ی هزینه، فاصله یا ظرفیت باشد.



شبکه‌ها در گراف‌ها

حال بیایید به مفهوم شبکه‌ها در گراف‌ها بپردازیم. وقتی در مورد شبکه‌ها در گراف‌ها صحبت می‌کنیم، معمولاً به شبکه‌های جریان اشاره داریم. این شبکه‌ها ابزاری قدرتمند برای مدل‌سازی و تحلیل جریان منابع (مانند آب، اطلاعات یا کالا) در یک سیستم هستند. در یک شبکه‌ی جریان، هر یال دارای یک ظرفیت (Capacity) است که حداکثر میزان جریانی را که می‌تواند از آن عبور کند، مشخص می‌کند. همچنین، در یک شبکه‌ی جریان، معمولاً دو راس ویژه وجود دارد: منبع (Source) که جریان از آن شروع می‌شود و مقصد (Sink) که جریان به آن ختم می‌شود. (تعداد منابع می‌تواند بیشتر از یکی هم باشد اما می‌توان یک منبع قرار داد که با وزن بی‌نهایت به بقیه‌ی منابع متصل است و تعداد آن‌ها را به یک منبع کاهش داد. برای مقصد هم به همین صورت است.)



مسائل شار بیشینه و برش کمینه

در شبکه‌های جریان، دو مسئله‌ی کلیدی و مرتبط به هم وجود دارد: مسئله‌ی شار بیشینه (Max Flow) و مسئله‌ی برش کمینه (Min Cut).

مسئله‌ی شار بیشینه:

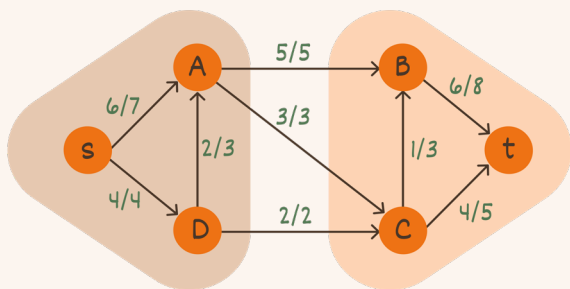
هدف از این مسئله، یافتن حداکثر میزان جریانی است که می‌توان از منبع به مقصد در شبکه ارسال کرد، به طوری که محدودیت ظرفیت هیچ یالی نقض نشود. به عبارت دیگر، می‌خواهیم بیشترین بهره‌وری را از شبکه‌ی جریان داشته باشیم.

مسئله‌ی برش کمینه:

یک برش در شبکه‌ی جریان، به نوعی تقسیم‌بندی راس‌های شبکه به دو مجموعه (مجموعه‌ی شامل منبع و مجموعه‌ی شامل مقصد) است، به طوری که یال‌های بین این دو مجموعه، مسیر جریان را قطع می‌کنند. ظرفیت یک برش، مجموع ظرفیت یال‌هایی است که از مجموعه‌ی منبع به مجموعه‌ی مقصد می‌روند. مسئله‌ی برش کمینه، یافتن برشی با کمترین ظرفیت ممکن است. به عبارت دیگر، می‌خواهیم گلوگاه شبکه‌ی جریان را پیدا کنیم که با کمترین هزینه، می‌توان جریان را متوقف کرد.

قضیه شار بیشینه-برش کمینه

شاید جالب باشد بدانید که مسئله‌ی شار بیشینه و برش کمینه، به طور عمیقی با هم مرتبط هستند. قضیه‌ی شار بیشینه-برش کمینه بیان می‌کند که مقدار شار بیشینه در یک شبکه‌ی جریان، دقیقاً برابر با ظرفیت کمینه‌ی یک برش در آن شبکه است! این قضیه نه تنها یک نتیجه‌ی نظری زیباست، بلکه راه را برای حل هر دوی این مسائل به طور همزمان باز می‌کند.



کاربردهای مسئله‌ی شار بیشینه

برنامه‌ریزی حمل و نقل:

بهینه‌سازی جریان ترافیک در شبکه‌های جاده‌ای، خطوط هوایی یا لوله‌های انتقال نفت و گاز.

شبکه‌های کامپیوتری:

مدیریت پهنای باند و بهینه‌سازی جریان داده‌ها در شبکه‌های اینترنتی.

مسائل تخصیص منابع:

تخصیص بهینه‌ی منابع محدود به متقاضیان مختلف، مانند تخصیص پهنای باند، نیروی کار یا بودجه.

تطبیق (Matching):

یافتن تطبیق‌های بیشینه در مسائل تخصیص، مانند تخصیص دانشجویان به پروژه‌های تحقیقاتی یا تخصیص کارمندان به وظایف.

کاربردهای مسئله برش کمینه

تحلیل آسیب‌پذیری شبکه:

شناسایی نقاط ضعف و گلوگاه‌های حیاتی در شبکه‌های ارتباطی، حمل و نقل یا زیرساخت‌های حیاتی، به منظور افزایش استحکام و پایداری آن‌ها.

بخش‌بندی تصویر:

(Image Segmentation) جداسازی اشیاء از پس‌زمینه در تصاویر دیجیتال، با استفاده از مفاهیم برش کمینه در گراف‌های مرتبط با تصویر.

طراحی مدارهای الکترونیکی:

تقسیم‌بندی و بهینه‌سازی اجزای مدارهای الکترونیکی برای کاهش پیچیدگی و بهبود عملکرد.

الگوریتم یافتن شار بیشینه

برای حل مسئله شار بیشینه، الگوریتم‌های مختلفی وجود دارد. یکی از الگوریتم‌های پایه‌ای، الگوریتم فورد-فالکرسون (Ford-Fulkerson) است. این الگوریتم با رویکردی ساده، به تدریج جریان شبکه را افزایش می‌دهد تا به شار بیشینه دست یابد.

مفاهیم کلیدی

مسیر افزایشی: (Augmenting Path) یک مسیر از منبع به مقصد در شبکه جریان است که هنوز ظرفیت خالی برای افزایش جریان دارد. به عبارت دیگر، یال‌های پیشرو در مسیر هنوز به ظرفیت کامل خود نرسیده‌اند و یال‌های بازگشتی در مسیر دارای ظرفیت غیرصفر هستند.

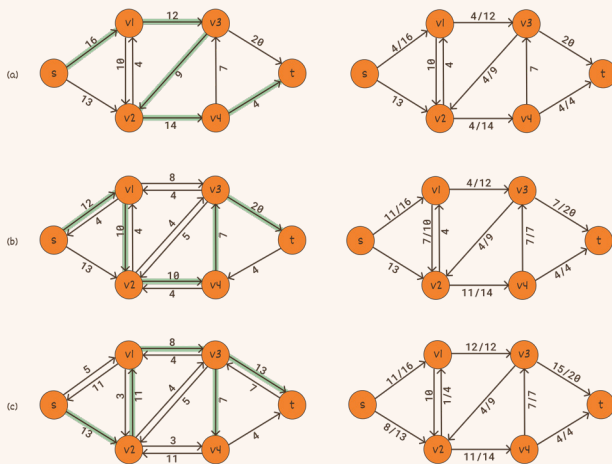
گراف باقی‌مانده: (Residual Graph) گرافی است که وضعیت باقی‌مانده‌ی ظرفیت یال‌ها را پس از اعمال جریان نشان می‌دهد. برای هر یال اصلی با ظرفیت c و جریان f ، در گراف باقی‌مانده دو یال وجود دارد: یال پیشرو با ظرفیت $c-f$ که نشان‌دهنده‌ی ظرفیت باقی‌مانده برای افزایش جریان در جهت اصلی است. یال بازگشتی با ظرفیت f که نشان‌دهنده‌ی امکان کاهش جریان در جهت عکس است.

شرح الگوریتم فورد-فالکرسون

- ۱. شروع با جریان صفر:** در ابتدا، جریان عبوری از تمام یال‌های شبکه را صفر در نظر بگیرید.
- ۲. جستجوی مسیر افزایشی:** در گراف باقی‌مانده، با استفاده از روش جستجوی عمق اول (DFS) یا جستجوی سطح اول (BFS)، به دنبال یک مسیر افزایشی از منبع به مقصد بگردید.
- ۳. افزایش جریان:** اگر یک مسیر افزایشی پیدا شد، میزان جریانی که می‌توان در طول این مسیر افزایش داد را محاسبه کنید. این میزان برابر است با کمترین ظرفیت باقی‌مانده در بین یال‌های مسیر. سپس، جریان را به این میزان در طول مسیر افزایشی افزایش دهید.
- ۴. به‌روزرسانی گراف باقی‌مانده:** پس از افزایش جریان، گراف باقی‌مانده را با توجه به تغییرات ظرفیت یال‌ها به‌روزرسانی کنید. ظرفیت یال‌های پیشرو در مسیر افزایشی کاهش می‌یابد و ظرفیت یال‌های بازگشتی افزایش می‌یابد.
- ۵. تکرار:** مراحل ۲ تا ۴ را تکرار کنید تا زمانی که دیگر هیچ مسیر افزایشی از منبع به مقصد در گراف باقی‌مانده وجود نداشته باشد.
- ۶. شار بیشینه:** هنگامی که هیچ مسیر افزایشی یافت نشد، جریان فعلی در شبکه، شار بیشینه است. مقدار شار بیشینه برابر با مجموع جریان خروجی از منبع (یا

مجموع جریان ورودی به مقصد) خواهد بود.

مثال اجرای الگوریتم



راس‌ها: راس‌ها با نام‌های s (منبع)، v_1 ، v_2 ، v_3 ، v_4 و t (مقصد) مشخص شده‌اند.

یال‌ها: یال‌ها جهت‌دار هستند و بر روی هر یال، دو عدد مشخص شده است: عدد اول (قبل از اسلش) نشان‌دهنده جریان فعلی عبوری از یال است (در ابتدا صفر است) و عدد دوم (بعد از اسلش) نشان‌دهنده‌ی ظرفیت یال است. برای مثال، یال از s به v_1 با $16/4$ نشان داده شده است، به این معنی که جریان فعلی ۴ و ظرفیت ۱۶ است.

مرحله (a)

مسیر افزایشی: در این مرحله، یک مسیر افزایشی از منبع s به مقصد t در گراف باقی‌مانده پیدا شده است. این مسیر با خطوط ضخیم‌تر در گراف سمت چپ مشخص شده است:

$$s \rightarrow v_1 \rightarrow v_3 \rightarrow v_2 \rightarrow v_4 \rightarrow t$$

میزان افزایش جریان: حداقل ظرفیت باقی‌مانده در طول مسیر افزایشی برابر با ۴ است (حداقل بین ۱۶، ۱۲، ۹، ۱۴، ۷). بنابراین، جریان به میزان ۴ واحد در طول این مسیر افزایش می‌یابد.

مرحله (b)

مسیر افزایشی جدید: مسیر افزایشی جدید:

$$s \rightarrow v_1 \rightarrow v_2 \rightarrow v_4 \rightarrow v_3 \rightarrow t$$

این مسیر نیز با خطوط ضخیم‌تر در گراف سمت چپ نشان داده شده است.

میزان افزایش جریان: حداقل ظرفیت باقی‌مانده در طول مسیر افزایشی برابر با ۷ است (حداقل بین ۱۲، ۱۰، ۱۰، ۷، ۲۰). بنابراین، جریان به میزان ۷ واحد در طول این مسیر افزایش می‌یابد.

مرحله c

مسیر افزایشی جدید: یک مسیر افزایشی دیگر پیدا شده است:

$$s \rightarrow v_2 \rightarrow v_1 \rightarrow v_3 \rightarrow t$$

این مسیر نیز با خطوط ضخیم تر مشخص شده است.

میزان افزایش جریان: حداقل ظرفیت باقی مانده در طول مسیر افزایشی برابر با ۸ واحد است (حداقل بین ۱۳، ۱۱، ۸، ۱۳). بنابراین، جریان به میزان ۸ واحد در طول این مسیر افزایش می یابد.

پایان الگوریتم

گراف سمت راست پایانی (گراف باقی مانده): در گراف سمت راست مرحله (c) دیگر هیچ مسیر افزایشی از منبع (s) به مقصد (t) در گراف باقی مانده وجود ندارد. بنابراین، الگوریتم به پایان رسیده و جریان فعلی در شبکه، شار بیشینه است.

شار بیشینه: مقدار شار بیشینه را می توان با جمع کردن جریان های خروجی از منبع (s) یا جریان های ورودی به مقصد (t) کرد. مقدار شار بیشینه $19 = 8 + 11 = 10 + 9$ در این مثال است.

پیچیدگی و الگوریتم های بهینه تر

الگوریتم فورد-فالکرسون یک الگوریتم پایه ای و قابل فهم است، اما ممکن است در برخی موارد کارایی بالایی نداشته باشد. پیچیدگی زمانی این الگوریتم به ظرفیت یال ها بستگی دارد و در بدترین حالت می تواند بسیار زیاد باشد. الگوریتم های بهینه تری مانند الگوریتم ادموندز-کارپ (Edmonds-Karp) با استفاده از جستجوی سطح اول (BFS) برای یافتن مسیرهای افزایشی، پیچیدگی زمانی بهتری ارائه می دهند و برای اکثر مسائل عملی، کارآمدتر هستند.

پیچیدگی زمانی الگوریتم فورد-فالکرسون

تعداد تکرار حلقه ی اصلی (یافتن مسیر افزایشی): الگوریتم فورد-فالکرسون در هر مرحله، یک مسیر افزایشی از منبع به مقصد در گراف باقی مانده پیدا می کند و جریان را در طول آن مسیر افزایش می دهد. این فرایند تا زمانی تکرار می شود که دیگر هیچ مسیر افزایشی وجود نداشته باشد. در بدترین حالت، ممکن است تعداد زیادی مسیر افزایشی وجود داشته باشد که الگوریتم مجبور به پیدا کردن و افزایش جریان در طول آن ها شود.

زمان لازم برای یافتن یک مسیر افزایشی:

برای پیدا کردن یک مسیر افزایشی در هر مرحله، معمولاً از الگوریتم های جستجوی گراف مانند جستجوی

عمق اول (DFS) یا جستجوی سطح اول (BFS) استفاده می شود. در یک گراف با V راس و E یال، زمان اجرای این جستجو ها معمولاً در حدود $O(V+E)$ است که می توان آن را به صورت تقریبی $O(E)$ در نظر گرفت، زیرا معمولاً تعداد یال ها بیشتر از تعداد راس ها است (مخصوصاً در شبکه های متراکم).

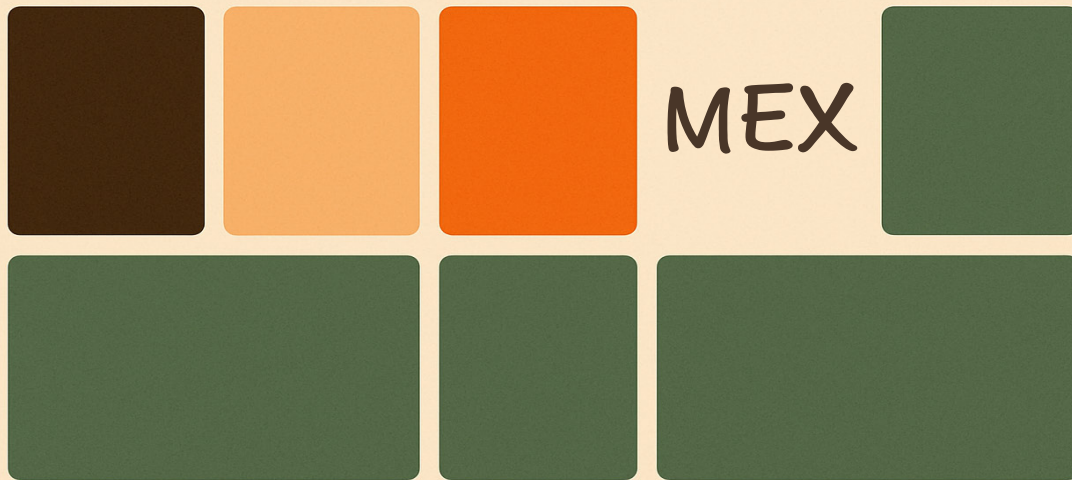
بدترین حالت پیچیدگی

در بدترین حالت برای الگوریتم فورد-فالکرسون، تعداد مسیرهای افزایشی که الگوریتم ممکن است پیدا کند، می تواند به مقدار شار بیشینه (Max Flow) بستگی داشته باشد. فرض کنید f مقدار شار بیشینه باشد. در بدترین سناریو، هر بار که الگوریتم یک مسیر افزایشی پیدا می کند، فقط جریان را به اندازه ۱ واحد افزایش می دهد. در این صورت، الگوریتم ممکن است تا f مرحله تکرار شود.

بنابراین، در بدترین حالت، پیچیدگی زمانی الگوریتم فورد-فالکرسون می تواند به صورت تقریبی $O(f \times E)$ باشد.

الگوریتم های دیگری برای پیدا کردن شار بیشینه نیز وجود دارند که پیچیدگی زمانی بهتری نسبت به این الگوریتم دارند. مانند الگوریتم ادموندز-کارپ که دارای پیچیدگی $O(V \times E^2)$ است.





میزهایی در مورد MEX



پویا رحیم‌پور

دانشجوی مقطع کارشناسی مهندسی کامپیوتر دانشگاه اصفهان

مسئله‌ی اول) $n \leq 2 \times 10^5$ عدد در آرایه a داریم $(a[i] \leq 10^9)$. می‌خواهیم MEX این اعداد را به دست آوریم.^۱

(حل)

- جواب نمی‌تونه بزرگتر از n باشه. (چرا؟)
 - پس می‌تونیم همه اعداد $i \in [0, n]$ چک کنیم. در این‌جا چند راه‌حل ممکنه. یکی اینکه از set استفاده کنیم. یک set از آرایه درست می‌کنیم. سپس برای هر عدد اگر در set نبود آن عدد MEX است. $O(n \log n)$
 - یک راه‌حل معقول‌تر مرتب کردن آرایه و پیمایش روی اون هست. اول مقدار MEX را برابر با صفر قرار می‌دهیم و اگر در یک مرحله عددی برابر با آن بود MEX را یکی زیاد می‌کنیم. اگر مساوی بود کاری نداریم و جلو می‌رویم. اگر کمتر بود همان عدد را به عنوان MEX برمی‌گردانیم: $O(n \log n)$

سوال) چطور مرتبه زمانی راه‌حل را به $O(n)$ کاهش دهیم؟

در دنیای الگوریتم‌ها و برنامه‌نویسی، همیشه با چالش‌های جدیدی روبرو هستیم که ممکنه در نگاه اول پیچیده به نظر بیان. بهترین راه برای یاد گرفتن این مفاهیم اینه که خودتون دست به کار بشید و مسائل رو حل کنید. اینجوری نه تنها درک بهتری از الگوریتم‌ها پیدا می‌کنید بلکه مسیر حل اون‌ها رو هم خودتون تجربه می‌کنید. تصمیم گرفتیم به جای این‌که به توضیح الگوریتم‌ها و داده‌ساختارهای پیچیده بپردازیم، چند مسئله ساده‌تر رو انتخاب کنیم و روندی که خودم برای حل اون‌ها طی کردم رو توضیح بدم. هدفم اینه که این مطالب نه تنها برای شما جذاب باشه بلکه بتونید با این مسائل، راهی برای درک بهتر الگوریتم‌ها و انگیزه برای مطالعه بیشتر پیدا کنید. یکی از این مسائل که در اینجا بهش می‌پردازیم، MEX هست که احتمالا در خیلی از سوالات الگوریتمی باهاش روبرو شدین. چرا MEX؟ دلیل خاصی نداشت صرفا چون اخیرا توی سوال‌ها زیاد دیدمش (:)

قبل از اینکه شروع کنیم پیشنهاد می‌کنم قبل از خوندن راه‌حل هر قسمت سعی کنید کد سوال رو بنویسید یا حداقل کمی به اون فکر کنید. وگرنه باید از خودتون بپرسید اصلا چرا دارید چشمتون را با این متن خسته می‌کنید؟ سعی کردم راه‌حل‌ها رو طوری بنویسم که بعد از هر جمله هم بتونید متوقف بشید و فکر کنید.

۱. MEX: مخفف Minimum Excluded هست. یعنی اولین عدد صحیح نامنفی که در لیست ما نیست.

این مسیر محتمل‌تره. به علاوه یک چالشی که داریم اعداد تکراری است و اینکه حذف کردن یک عدد همه نمونه‌های اون رو از لیست حذف نمی‌کنه بلکه تعداد اون رو یکی کمتر می‌کنه.

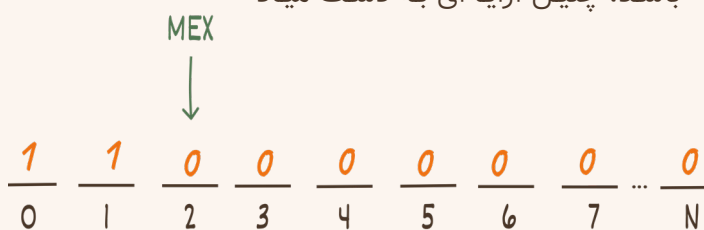
حل ۱)

- مثل قبل هر عددی که بزرگ‌تر از n باشه مهم نیست.
- ما به جایگاه‌های خالی علاقه‌مندیم نه جایگاه‌های پر.
- تعداد جایگاه‌های خالی مهم بیش‌تر از $1+n$ نیست. (برای اعداد $[0, n]$)
- با سه راهنمایی قبلی می‌شه فهمید که به داده‌ساختاری نیاز داریم که جایگاه‌های خالی رو به صورت مرتب برای ما نگه‌داره. و این جایگاه‌ها می‌تونن اضافه بشن یا حذف بشن. احتمالا می‌دونید که set همه این کارها رو از $O(\log n)$ انجام می‌ده.

سوال) چگونه اعداد تکراری را مدیریت کنیم؟

کافیه تعداد هر عدد رو نگه‌داریم. به محض اینکه تعداد اون به صفر رسید باید به set اضافه بشه چون یک جایگاه خالی جدید و وقتی که عددی اضافه شد که قبلا تعداد اون صفر بود باید اون عدد رو از set حذف کنیم. برای نگه داشتن تعداد اعداد می‌تونید از یک map استفاده کنید ولی دقت کنید که چون اعداد بزرگ مهم نیستن می‌شه این کار رو حتی با یک آرایه انجام داد. (پیشنهاد می‌کنم همیشه اگر ممکن هست map یا set رو با آرایه جایگزین کنید) غیر از روش اول یک روش دیگه هست که از یک خاصیت خوب MEX استفاده می‌کنه.

ایده) اگر x یک عدد کوچک‌تر از MEX باشد، تعداد اعداد متمایز کوچک‌تر یا مساوی x برابر با x است؛ و اگر x بزرگ‌تر یا مساوی MEX باشد، این تعداد اکیدا کم‌تر از x است. بنابراین MEX اولین عددی است که تعداد اعداد متمایز کوچک‌تر یا مساوی آن اکیدا کم‌تر از آن است. اگر یک آرایه‌ی $boolean$ برای اعداد 1 تا n بسازیم و در آن جایگاه 1 قرار دهیم اگر و تنها اگر تعداد اعداد متمایز کوچک‌تر یا مساوی آن برابر با خود عدد باشد، چنین آرایه‌ای به دست میاد



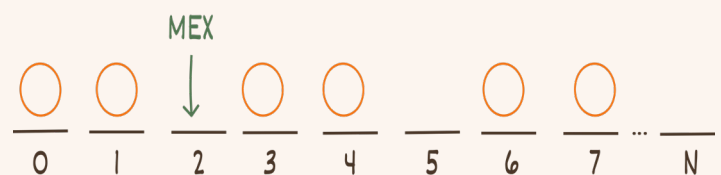
احتمالا می‌دونید که روی چنین توابعی می‌شه برای پیدا کردن اولین جایگاه ۰ از Binary Search استفاده کرد. گرچه در این مسئله زیاد مهم نیست ولی دقت کنید که در Binary Search، ایجاد کردن کل آرایه هیچ‌وقت

اعدادی که از n بزرگ‌تر باشند مهم نیستند. (چرا؟) پس در راه اول می‌شد به جای set از یک آرایه $boolean$ با طول $1+n$ استفاده کرد که نشان می‌دهد آیا آن عدد در آرایه وجود دارد یا نه.

خب تا اینجا احتمالا همه‌چیز خیلی ساده است. بیایید مسئله را کمی جالب‌تر کنیم.

مسئله دوم) در این مسئله $q \leq 2 \times 10^5$ پرسرمان داریم. هر پرسرمان از سه نوع است. نوع ۱) عدد x را به لیست اضافه کنید. نوع ۲) MEX اعداد لیست را در این لحظه به دست آورید. نوع ۳) عدد $x \leq 10^9$ را که تضمین می‌شود قبلا حداقل یکی از آن در لیست بوده از لیست حذف کنید. که در آن‌ها به علاوه تضمین می‌شود که تعداد اعداد لیست در هر لحظه از $n \leq 2 \times 10^5$ بیش‌تر نمی‌شود.

توضیح) قرار نیست حل مسئله همیشه یک روال یکنواخت و روتین داشته باشه (حداقل این‌طور به نظر میاد). مثلا ایده‌ی اولی که برای این سوال داشتم این بود که به نحوی فاصله‌های متصل رو سریع‌تر پیمایش کنم. مثلا هر جایگاه به اولین جایگاه خالی مساوی یا بعد از خودش اشاره کنه، ولی انگار ادغام و شکستن چنین موجوداتی حداقل در زمان مناسب ساده نیست. قبل از هر چیزی بهتره اون چیزی که در تصورم هست رو نشون بدم. این مفیده ولی شما اون چیزی رو که من یا هر کسی بهتون میگه یاد نمی‌گیرید. بلکه اون چیزی رو یاد می‌گیرید که بهش فکر می‌کنید. (حتی دیدن این شکل کافی نیست و خوبه که بتونید حالت‌های مختلف دیگه ای رو هم تصور کنید). تصور من از اعداد به این شکله:



ما به اولین جای خالی علاقه‌مندیم. و اینکه اضافه کردن یا برداشتن دایره‌ها چطور جای اون رو تغییر می‌ده. مثلا اینجا اگه عدد ۲ رو اضافه کنیم، $MEX = 5$ می‌شه و اگه بعد از اون ۱ رو حذف کنیم، $MEX = 1$ میشه. همین‌جا به نظر میاد اگه مسئله فقط اعداد رو حذف می‌کرد خیلی ساده می‌شد. چون اگر عددی که حذف می‌کنید کمتر از MEX باشه همون چیزی که حذف می‌کنید، MEX جدید است. در غیر این‌صورت مهم نیست چی رو حذف می‌کنید.

روشی که در ادامه می‌گم اولین روشی نیست که به ذهن من رسید ولی فکر می‌کنم اگر شما با داده‌ساختارهایی که برای Range Query استفاده می‌شن آشنا نباشید

نیاز نیست و کافیه مقدار تابع یکنوا را در تقریباً $\log n$ نقطه حساب کرد. اگر این جمله براتون معنی نداره احتمالا به اندازه کافی از Binary Search مسئله حل نکرده‌اید (و وقت خوبی که شروع کنید). اما غیر از اون نیاز داریم تعداد اعداد متمایز کوچکتر مساوی هر عدد در این بازه رو با مرتبه زمانی خوب به‌روز نگه داریم. مثلا اگر یک عدد اضافه/حذف بشه باید همه اعداد بزرگتر مساوی آن را یک واحد زیاد/کم کنیم.

(حل ۲)

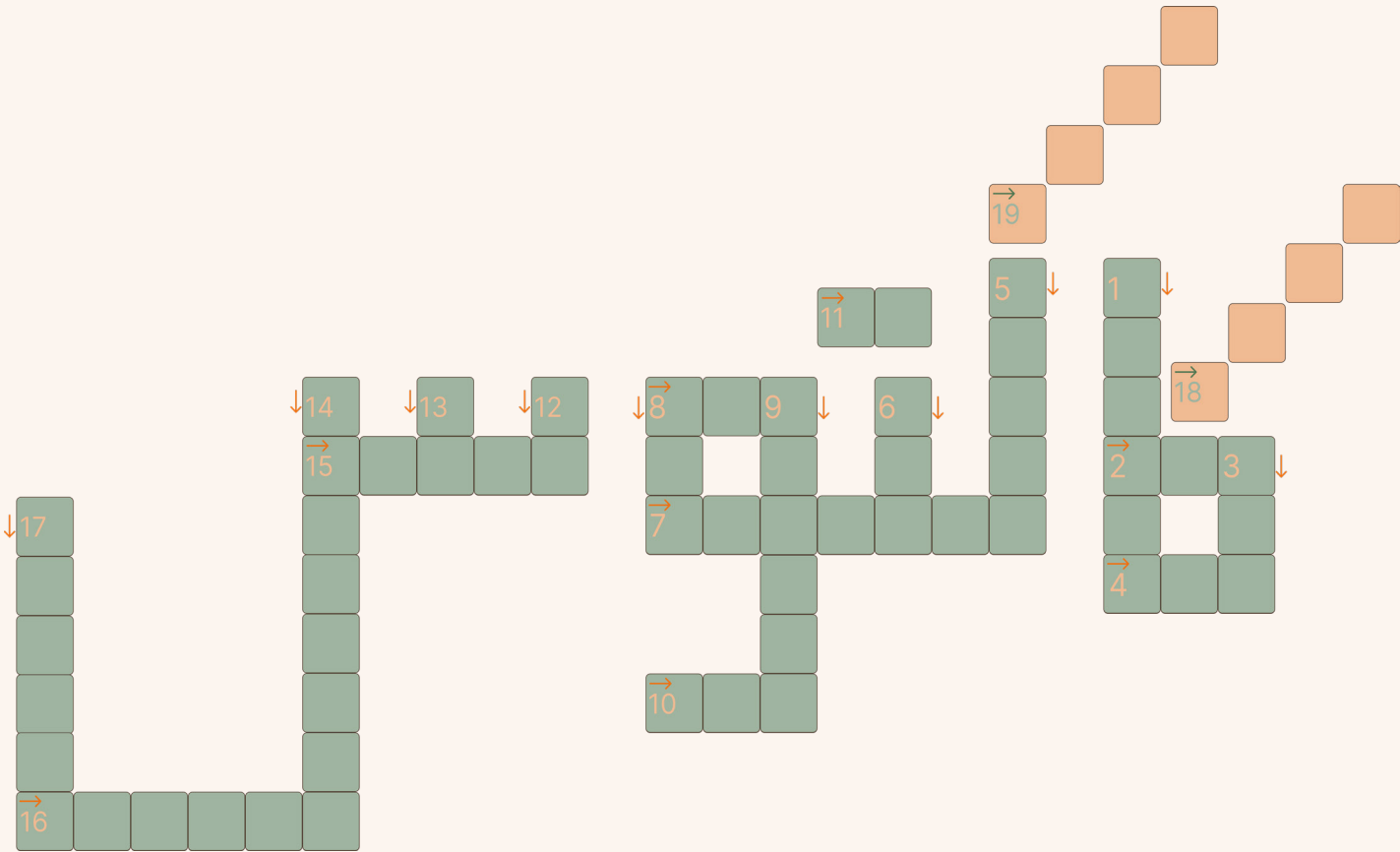
- از یک Fenwick Tree یا Segment tree برای عمل آخر استفاده کنید. (اگر این داده‌ساختار ها رو نمی‌شناسید ولی بقیه این متن بدیهی به نظر می‌رسه وقت خوبی که حداقل راجع به اولی بخونید).
- برای پیدا کردن اولین $\circ$ از Binary Search استفاده کنید.

امیدوارم از این متن استفاده برده باشید. پیشنهاد می‌کنم حتما هر دو مسئله رو پیاده‌سازی کنید. (باور کنید اگر زیاد مسئله حل نکرده‌اید فاصله چیزی که فکر می‌کنید تا چیزی که پیاده‌سازی می‌کنید کم نیست و خیلی وقت‌ها شاید فکر کنید چیزی را می‌فهمید ولی در زمان پیاده‌سازی متوجه می‌شوید که خیلی چیزها رو نمی‌دیدید).^۲

^۲. متأسفانه یا خوش‌بختانه درجین نوشتن متن متوجه شدم که تقریباً همه‌ی چیزی که گفتم در وب‌سایت cp-algorithm هست :) با این حال امیدوارم این نسخه از مطالب رو دوست داشته باشید. پس شاید بهتر باشه این لینک رو هم چک کنید.
<https://cp-algorithms.com/sequences/mex.html>

پ.ن: برای آماده کردن این متن چندتا ایده داشتم. مثلا یک داده‌ساختاری که خودم هم نمی‌دونم رو مطالعه کنم و راجع به اون بنویسم یا یک الگوریتم معروف رو دوباره توضیح بدهم. ولی در نهایت به این نتیجه رسیدم که همه این چیزها توسط افراد خیلی قوی‌تر از من قبلا نوشته شده‌اند و احتمالا خواندن چنین موضوعاتی از بلاگ یک LGM معقول‌تره.

گاہتوس پراس پراس



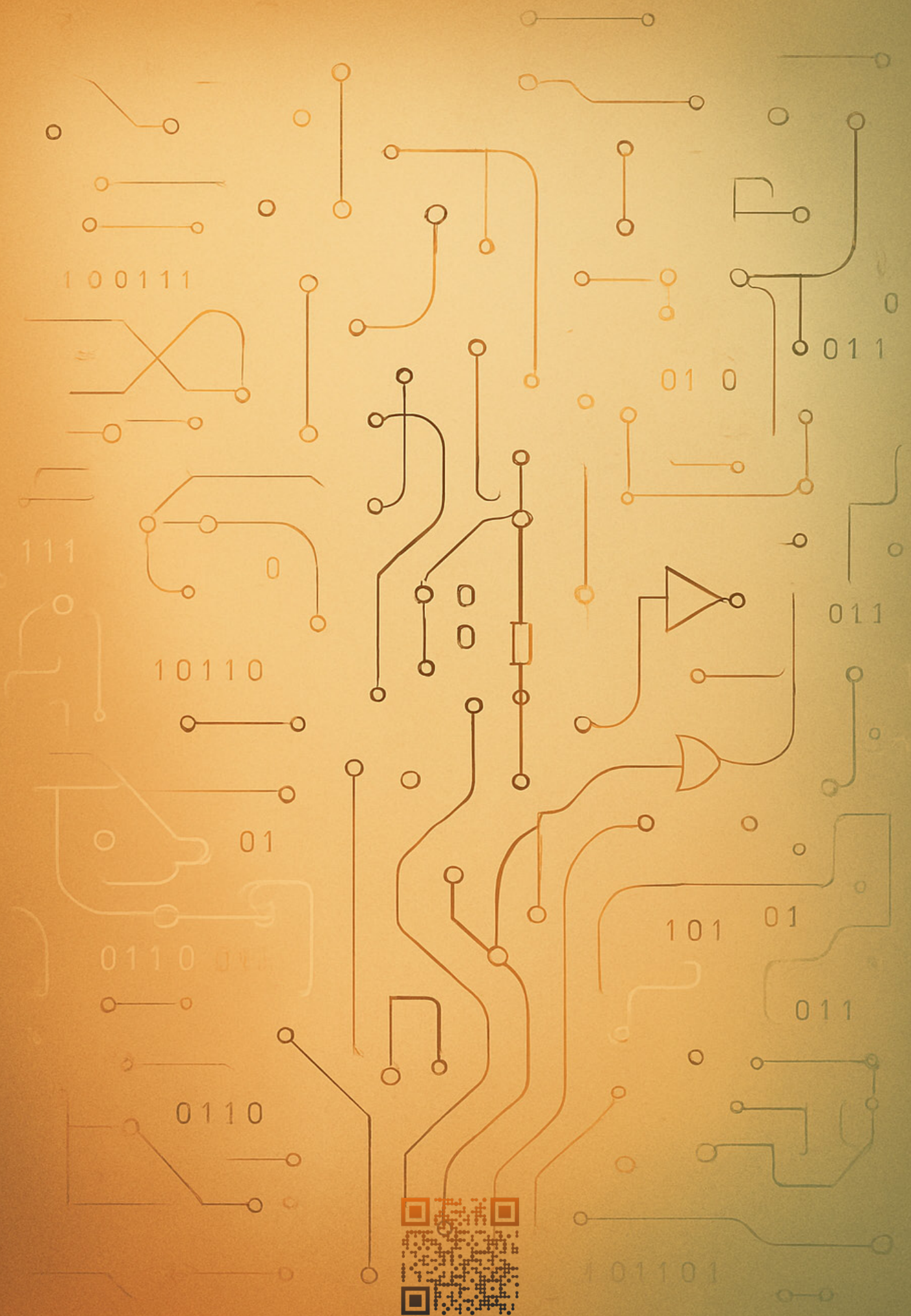
افقی:

۲. سیستم کنترل نسخه‌ای که توسعه‌دهندگان برای مدیریت تغییرات در کد منبع از آن استفاده می‌کنند.
۴. به پارامترهایی گفته می‌شود که هنگام فراخوانی تابع به آن ارسال می‌شوند.
۷. نوعی الگوریتم الهام گرفته از فرایند تکامل طبیعی
۸. مجموعه‌ای از توابع و کلاس‌ها که می‌توان آن را در پروژه‌های مختلف فراخوانی کرد.
۱۰. در پایگاه داده‌ها، از آن برای شناسایی یکتا و ارتباط بین جداول استفاده می‌شود.
۱۱. شاخه‌ای از علوم رایانه که هدف آن شبیه‌سازی توانایی‌های انسانی مانند یادگیری و تصمیم‌گیری است.
۱۵. الگوریتمی برای یافتن کوتاه‌ترین مسیر
۱۶. به ماتریس‌هایی با تعداد زیادی عنصر صفر گفته می‌شود.
۱۸. داده‌ساختاری سلسله‌مراتبی که شامل ریشه و گره‌های فرزند است.
۱۹. داده‌ساختاری درختی که از آن برای پیاده‌سازی صف اولویت استفاده می‌شود.

عمودی:

۱. ماشین رمزنگاری معروفی که در جنگ جهانی دوم استفاده می‌شد.
۳. در سیستم‌های کنترل نسخه، برای مشخص کردن یک نقطه‌ی خاص از تاریخچه‌ی پروژه مانند نسخه‌ی نهایی استفاده می‌شود.
۵. پایه‌ی اصلی الگوریتم‌ها که از گزاره‌ها و عملگرهای منطقی برای تصمیم‌گیری استفاده می‌کند.
۶. نوع داده‌ای در زبان‌های برنامه‌نویسی که برای ذخیره‌سازی اعداد صحیح استفاده می‌شود.
۸. ثبت خودکار اتفاقات و خطاها در نرم‌افزارها برای بررسی و رفع اشکال
۹. سیستم عددی مورد استفاده در رایانه‌ها
۱۲. یکی از عملگرهای منطقی که در صورت درست بودن حداقل یکی از ورودی‌ها، خروجی را درست می‌کند.
۱۳. حوزه‌ای از دانش که به استفاده از فناوری برای پردازش اطلاعات می‌پردازد.
۱۴. نمادی برای ذخیره‌سازی مقادیر که می‌توان آن را تغییر داد.
۱۷. نام یک ساختار گراف خاص که از قضا الهام‌بخش نام نشریه‌ای است که در حال مطالعه‌ی آن هستید!





اگر نقد، ایده یا تمایلی برای همکاری با انجمن
و شماره‌های بعدی نشریه دارید، از طریق اسکن
این بارکد به راحتی با ما در میان بگذارید.